Git & GitHub Complete Commands Guide with Purpose

---

1. Git Installation Verification

Command:

```
git --version
```

Why this is required?
Used to verify whether Git is installed successfully in the system.

How this is helpful in real-time?
Helps developers confirm Git setup before starting project work.

2. Configure Git Username

Command:

```
git config --global user.name "Sindhu"
```

Why this is required?
Used to configure Git username globally.

How this is helpful in real-time?
Helps identify who made code changes in project history.

3. Configure Git Email

Command:

```
git config --global user.email "abc@gmail.com"
```

Why this is required?
Used to configure Git email globally.

How this is helpful in real-time?
Helps associate commits with GitHub account.

4. Check Git Configuration

Command:

```
git config --list
```

Why this is required?
Displays all configured Git settings.

How this is helpful in real-time?
Helps verify username, email, and Git configurations.

5. Initialize Repository

Command:

```
git init
```

Why this is required?
Creates a new local Git repository inside the project folder.

How this is helpful in real-time?
Helps Git start tracking all project files and changes.

6. Check Repository Status

Command:

```
git status
```

Why this is required?
Shows modified, staged, committed, and untracked files.

How this is helpful in real-time?
Helps developers understand current project changes before commit.

7. Add Files to Staging Area

Command:

```
git add .
```

Why this is required?
Moves all changed files to the staging area.

How this is helpful in real-time?
Helps prepare files before committing changes into Git history.

8. Commit Changes

Command:

```
git commit -m "Initial Commit"
```

Why this is required?
Saves staged changes permanently into Git history.

How this is helpful in real-time?
Helps track project versions and allows rollback to older versions if needed.

9. View Commit History

Command:

```
git log --oneline
```

Why this is required?
Displays commit history in short format.

How this is helpful in real-time?
Helps quickly review previous project changes.

10. Clone Repository

Command:

```
git clone https://github.com/user/repo.git
```

Why this is required?
Downloads a remote GitHub repository into local machine.

How this is helpful in real-time?
Helps team members work on the same project locally.

11. Add Remote Repository

Command:

```
git remote add origin repositoryURL
```

Why this is required?
Connects local repository with GitHub repository.

How this is helpful in real-time?
Helps push and pull code between local and remote repositories.

12. Verify Remote Repository

Command:

```
git remote -v
```

Why this is required?
Displays configured remote repository URLs.

How this is helpful in real-time?
Helps verify correct GitHub repository connection.

13. Push Changes

Command:

```
git push
```

Why this is required?
Uploads local commits to remote GitHub repository.

How this is helpful in real-time?
Helps share latest code changes with team members.

14. Pull Changes

Command:

```
git pull
```

Why this is required?
Downloads latest code changes and merges them locally.

How this is helpful in real-time?
Helps keep local project updated with team changes.

15. Fetch Changes

Command:

```
git fetch
```

Why this is required?
Downloads latest remote changes without merging.

How this is helpful in real-time?
Helps review incoming changes safely before merge.

16. Create and Switch Branch

Command:

```
git checkout -b feature-login
```

Why this is required?
Creates and switches to a new branch.

How this is helpful in real-time?
Helps developers work independently without affecting main code.

17. Switch Branch

Command:

```
git switch feature-login
```

Why this is required?
Moves from one branch to another.

How this is helpful in real-time?
Helps developers work on multiple features easily.

18. Merge Branch

Command:

```
git merge feature-login
```

Why this is required?
Combines branch code into current branch.

How this is helpful in real-time?
Helps integrate completed features into main project.

19. Temporary Save Using Stash

Command:

```
git stash
```

Why this is required?
Temporarily stores uncommitted changes.

How this is helpful in real-time?
Helps switch branches without losing unfinished work.

20. Undo Local Changes

Command:

```
git restore file.txt
```

Why this is required?
Removes local modifications from a file.

How this is helpful in real-time?
Helps restore original file content if changes are unnecessary.

21. Reset Commit

Command:

```
git reset --soft HEAD~1
```

Why this is required?
Removes last commit while keeping code changes.

How this is helpful in real-time?
Helps correct commit mistakes without losing work.

22. Revert Commit

Command:

```
git revert commitID
```

Why this is required?
Creates a new commit that reverses previous commit changes.

How this is helpful in real-time?
Helps safely undo commits in shared repositories.

23. Delete Branch

Command:

```
git branch -d feature-login
```

Why this is required?
Deletes local branch after merge completion.

How this is helpful in real-time?
Helps maintain clean repository structure.

24. Compare Code Changes

Command:

```
git diff
```

Why this is required?
Shows differences between modified and committed code.

How this is helpful in real-time?
Helps review code changes before commit.

25. Cherry Pick

Command:

```
git cherry-pick commitID
```

Why this is required?
Copies a specific commit from another branch.

How this is helpful in real-time?
Helps move only required changes between branches.

26. Rebase

Command:

```
git rebase main
```

Why this is required?
Moves branch commits on top of updated main branch.

How this is helpful in real-time?
Helps maintain clean linear project history.

27. Delete Remote Branch

Command:

```
git push origin --delete feature-login
```

Why this is required?
Deletes branch from remote repository.

How this is helpful in real-time?
Helps clean unused branches from GitHub.

28. Create Tag

Command:

```
git tag v1.0
```

Why this is required?
Creates version tags in Git.

How this is helpful in real-time?
Helps identify important releases easily.

29. Push Tag

Command:

```
git push origin v1.0
```

Why this is required?
Uploads created tag to GitHub.

How this is helpful in real-time?
Helps share release versions with team members.

30. Git Ignore

Command:

```
.gitignore
```

Why this is required?
Prevents unnecessary files from getting tracked.

How this is helpful in real-time?
Helps avoid pushing node_modules, reports, secrets, and temp files.

31. Amend Last Commit

Command:

```
git commit --amend -m "Updated Message"
```

Why this is required?
Updates previous commit message or changes.

How this is helpful in real-time?
Helps fix commit mistakes quickly.

32. Blame Command

Command:

```
git blame file.txt
```

Why this is required?
Shows who modified each line in a file.

How this is helpful in real-time?
Helps identify code ownership and debugging history.

33. Clean Untracked Files

Command:

```
git clean -f
```

Why this is required?
Removes untracked files from project.

How this is helpful in real-time?
Helps clean unnecessary temporary files.